

High-Level Design

Prior Authorization Multi-Agent Platform

**DEMONSTRATION SYSTEM — SYNTHETIC DATA — NOT FOR
CLINICAL
OR COVERAGE USE**

Document type: High-Level Design (HLD)

Version 1.1

Audience: architects, engineering leads, technical stakeholders

1. Purpose & Scope

This document describes the high-level architecture of the Prior Authorization Multi-Agent Platform: what problem it solves, how its major components fit together, the technology choices behind it, and the key design decisions that shape its behavior. It is written for readers who need to understand the system's shape before diving into implementation detail (covered separately in the companion Low-Level Design document).

This is a demonstration platform built on entirely synthetic patients and synthetic payer policies. It is not connected to any real payer, EHR, or claims system, and no output from it should be treated as an actual coverage determination.

2. Problem Statement

Prior authorization (PA) is one of the highest-friction workflows in healthcare administration. A provider requests a procedure, medication, or piece of equipment on behalf of a patient; a payer must check the request against coverage criteria — BMI thresholds, step-therapy requirements, documentation requirements, exclusions — before approving it. In practice this process is slow, often opaque to the provider and patient, and prone to inconsistent manual interpretation of policy text. Delays and unclear denials directly translate into delayed or denied patient care.

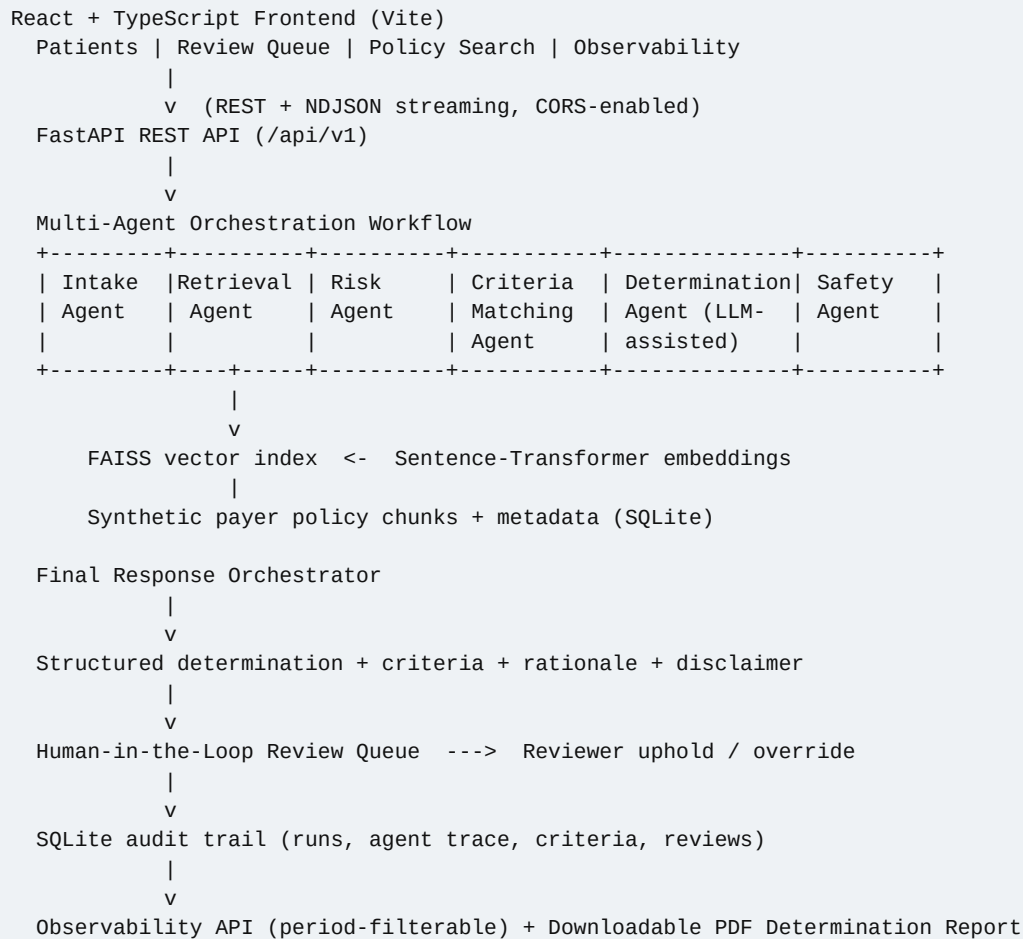
The platform demonstrates an alternative shape for this workflow:

- Every criterion in the applicable policy is evaluated individually and explicitly, not just a single opaque yes/no.
- The actual coverage decision is produced by a deterministic, reproducible rules engine — never by an LLM — so the same inputs always produce the same decision.
- A large language model is used only to explain a decision that has already been made, never to make it.
- No determination is ever presented as final without passing through a human clinical reviewer.
- Every step — every agent's input and output, every criterion evaluated, every review decision — is captured in a durable, queryable audit trail.

3. System Overview

The platform is a two-tier web application: a React single-page frontend and a Python/FastAPI backend. The backend runs a sequential pipeline of six purpose-built agents over each incoming request, backed by a local retrieval-augmented generation (RAG) index of synthetic payer policy documents. The pipeline's output — an approve/deny/pend determination with full supporting evidence — is queued for mandatory human review before it is considered final. Every run is recorded for both compliance auditing and operational observability.

3.1 Architecture Diagram



3.2 Major Components

Component	Responsibility
Frontend (React/TypeScript)	Patient management, PA request submission (with history-informed diagnosis-code pre-fill), live pipeline visualization, review queue UI, observability dashboard, policy search.
FastAPI Backend	REST + streaming API surface; routes requests to services; owns the exception-to-HTTP-status mapping.
Multi-Agent Orchestration	Six-agent sequential pipeline that turns a request into an evidence-grounded determination.
RAG / Retrieval Layer	FAISS vector index over chunked synthetic payer policy documents; sentence-transformer embeddings.
LLM Integration (optional, pluggable)	Anthropic Claude or OpenAI call (provider selected via LLM_PROVIDER) that drafts the natural-language rationale and approval-path suggestion for an already-decided outcome; degrades gracefully to a template when unconfigured.
Persistence (SQLite)	Patients, guidelines/chunks, orchestration runs, per-agent trace, per-criterion audit records.

Component	Responsibility
Human-in-the-Loop Review	Durable review queue; uphold/override workflow; enforced before a determination is final.
Observability	Aggregated health, volume, outcome, per-agent reliability/latency, review throughput, and LLM token/cost metrics; scoped by a trailing 7d/30d/month/year window or full history.
Reporting	Server-rendered PDF determination report reflecting the stored audit trail.

4. Technology Stack

Layer	Technology
Backend language/framework	Python 3.11+, FastAPI, Uvicorn, Pydantic v2
Data layer	SQLAlchemy 2.0 ORM over SQLite
Vector search	FAISS (IndexFlatIP on normalized embeddings)
Embeddings	Sentence-Transformers (all-MiniLM-L6-v2, configurable)
LLM	Anthropic Claude API or OpenAI API (optional; provider selectable via LLM_PROVIDER; official Python SDKs for both)
PDF generation	ReportLab
Frontend	React 18 + TypeScript, built with Vite, React Router
Charts	Recharts (agent latency/reliability, cost trend) + hand-built stacked bars (outcome breakdowns)
Testing	Pytest + pytest-asyncio + FastAPI TestClient (backend); tsc + Vite build (frontend)
Deployment	Docker + docker-compose (backend + nginx-served frontend)

5. High-Level Data Flow

A single prior authorization request moves through the system as follows:

1. A user submits a structured request (requested service, category, diagnosis codes, urgency, prior treatments tried) for a specific patient via the frontend; diagnosis codes are pre-filled from the patient's documented history and remain editable.
2. The backend streams the request through six agents in sequence, each contributing to a shared workflow state.
3. The Guideline Retrieval Agent embeds the request and retrieves the single most relevant synthetic payer policy from the FAISS index.
4. The Criteria Matching Agent extracts every individual numbered criterion from that policy and evaluates each as MET, NOT_MET, or UNKNOWN against the patient/request.
5. The Determination Agent applies a deterministic rule to the criteria evaluation to decide approve, deny, or pend — then optionally asks the configured LLM provider to draft the rationale for that decision.

6. The Safety Agent independently re-derives what the determination should be from the same criteria and flags any mismatch, plus verifies every cited criterion is grounded in retrieved text.
7. The complete result is persisted (run record, per-agent trace, per-criterion audit rows) and placed into the human review queue.
8. A clinical reviewer opens the queue, inspects the full AI response, and either upholds or overrides the determination with a required rationale.
9. The final, reviewed outcome can be downloaded at any time as a PDF report and is reflected in the observability dashboard.

6. Deployment Architecture

The system ships as two containers orchestrated by docker-compose: a FastAPI backend (port 8000) and an nginx-served static build of the React frontend (port 3000). The backend downloads and caches its embedding model on first startup and idempotently builds its FAISS index and seeds demonstration patients if the database is empty. An optional Anthropic or OpenAI API key (selected via LLM_PROVIDER) can be supplied as environment variables to enable real LLM-drafted rationales; without either, the system runs fully offline with a deterministic templated rationale.

```
docker compose up --build
  backend -> http://localhost:8000  (Swagger: /docs)
  frontend -> http://localhost:3000
```

7. Key Design Decisions & Rationale

Decision	Rationale
The coverage decision is always deterministic; the LLM only drafts rationale text.	Keeps the actual outcome reproducible and auditable even when the LLM is unavailable, misconfigured, or wrong — a regulated-adjacent workflow cannot depend on a non-deterministic component for its core decision.
The LLM provider is pluggable (Anthropic or OpenAI) behind one internal call.	Avoids a hard dependency on a single model vendor for a component that never makes the actual decision, and keeps both call sites (rationale + approval-path suggestion) provider-agnostic.
Every completed determination requires human review before being final.	Mirrors real utilization-management practice and gives the system's own disclaimer (requires_clinician_review) an actual enforced mechanism instead of being just a flag.
A request is evaluated against exactly one dominant policy, not every policy that scored above the retrieval threshold.	Prevents unrelated policies (e.g. durable medical equipment criteria) from leaking into an unrelated request (e.g. imaging) purely because a chunk scored reasonably well.
Negated/exclusion-style criteria never resolve to a confident denial from lexical overlap alone.	A wrong guess in the 'deny' direction blocks needed care; the safer failure mode is to flag the criterion UNKNOWN and let a human resolve it.
Every agent's input/output and every criterion evaluated is persisted, not just the final answer.	Enables both the operator-facing observability dashboard (including its trailing-window filters) and root-cause debugging of any individual determination after the fact.
The frontend streams per-agent completion events (NDJSON) instead of waiting for the whole pipeline.	Makes the multi-agent orchestration genuinely visible to the user in real time rather than behind a single opaque spinner.

8. Non-Functional Considerations

8.1 Auditability & Compliance-style Traceability

Every run, every agent execution, every evaluated criterion, and every review decision is written to durable storage with timestamps. A determination's PDF report is a rendering of this stored audit trail, not a fresh computation, so what a reviewer saw is always reproducible.

8.2 Observability

System health, request volume, outcome mix, per-agent latency/reliability (avg and p95), review-queue throughput, override rate, and real LLM token/cost usage are all aggregated from the same audit tables and exposed via a dashboard with proper chart forms (not decorative pie charts) and a validated, CVD-safe color palette. The request/outcome/agent sections can be scoped to a trailing 7-day, 30-day, 1-month, or 1-year window (or left as full history) independently of the dedicated LLM-cost date filter, which has its own 7d/30d/month/year/custom range.

8.3 Security & Scope Boundaries

- No authentication/authorization layer exists — the platform is scoped for local/demo use, not multi-tenant production deployment.
- No real payer, EHR, or claims (X12 278 / FHIR PriorAuth) integration exists.
- All patient and policy data is synthetic and clearly labeled as such throughout the UI and generated documents.

8.4 Resilience

Agents never crash the workflow: a failure inside any single agent is caught, recorded with status "error" in its trace entry, and the pipeline continues with the remaining agents, degrading gracefully toward a pended determination with a safety flag rather than a hard failure.

9. Limitations & Future Roadmap

Current limitations

- Criteria matching uses lexical heuristics (with a few rule tiers for numeric/step-therapy/exclusion patterns), not a full clinical-NLP or logic-graph engine.
- The synthetic policy corpus is small (five documents) and not representative of real-world payer policy volume or complexity.
- No authentication, multi-tenancy, or real payer/EHR integration.
- LLM cost figures are illustrative estimates, not billing-accurate.
- Diagnosis-code pre-fill maps a small, controlled vocabulary of documented conditions to representative ICD-10 codes; it is not a full ICD-10 lookup/autocomplete service.

Roadmap

- A logic-aware criteria model with explicit AND/OR grouping between criteria.
- Deeper negation/conditional-criteria resolution beyond lexical overlap.
- Role-based access control, audit logging, and multi-tenant isolation for a real deployment.

- Frontend automated test coverage (component/E2E).
- Streaming the LLM-drafted rationale token-by-token, in addition to the existing per-agent pipeline streaming.
- A full ICD-10 typeahead/autocomplete for diagnosis codes beyond the current history-based pre-fill.

10. Glossary

Term	Meaning
PA	Prior Authorization — payer approval required before a service is covered.
Determination	The approve / deny / pend outcome produced by the platform for a request.
Criterion	One individually evaluable rule extracted from a payer policy document.
HITL	Human-in-the-Loop — the mandatory reviewer sign-off gate before a determination is final.
RAG	Retrieval-Augmented Generation — retrieving relevant text before generating/reasoning over it.
NDJSON	Newline-Delimited JSON — one JSON object per line, used for the streaming run endpoint.
Upheld / Overridden	Reviewer decision to keep the AI determination as-is, or replace it with their own.